PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

CONSULTAS DE BASE DE DATOS JARDINERIA

Presentado a: **Instructora Yoli Alexandra Muños Guevara**

Por Aprendiz: **Sebastián Alejandro Bolaños**

Ficha: 3312932

Competencia: **Frontend**

Resultado de aprendizaje: Diseñar e implementar la interfaz de usuario (Frontend) aplicando principios de arquitectura

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **20** de **09** del año **2026**

Tabla de Contenido

1. Objetivo de aprendizaje 3
2. Requerimientos funcionales 3
3. Investigación obligatoria: middleware en Express 6
4. Construcción del middleware propio 8
5. Códigos de estado HTTP y manejo de errores 11
6. Pruebas de la API 14
7. Documento técnico obligatorio: paso a paso 18

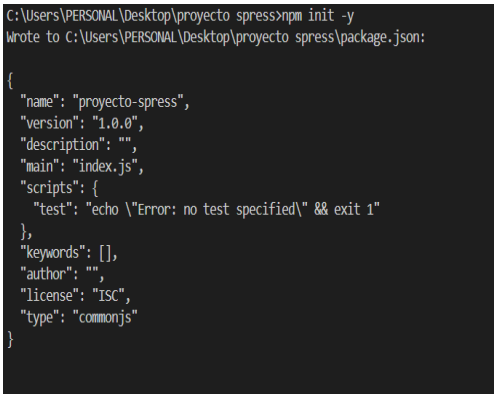
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

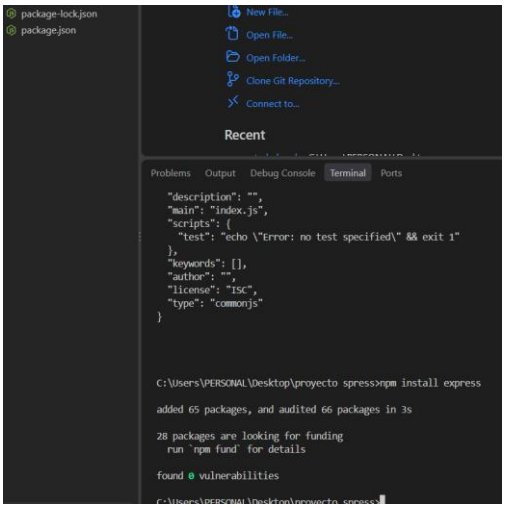
1. Objetivo de aprendizaje

Construir una API pequeña con Express, comprender el flujo de una petición HTTP y explicar cómo se aplican los middleware, la validación de datos, el manejo de errores, Nodemon y el control de versiones

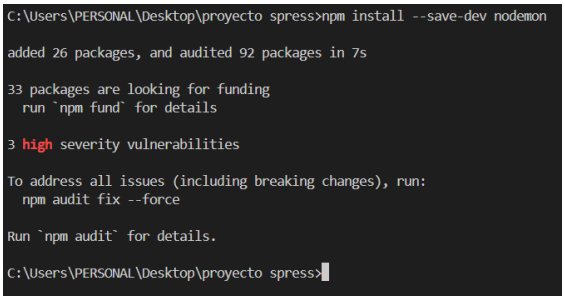
2. Requerimientos funcionales

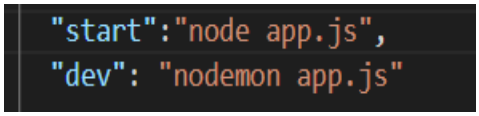
1. **Crear un proyecto nuevo de Node.js y Express.**





   - **npm init -y**: crea el archivo package.json.
   - **npm install express**: instala Express para crear el servidor y las rutas y trae un paquete lock.json
2. **Configurar una forma de ejecutar el servidor en modo normal y otra en modo desarrollo con Nodemon.**





npm start

Este comando ejecuta el servidor en modo normal.

npm run dev

Este comando lo ejecuta con Nodemon, que reinicia automáticamente el servidor cuando detecta cambios en el código.

3. **Crear una ruta GET /inicio que devuelva un mensaje de bienvenida en formato JSON.**



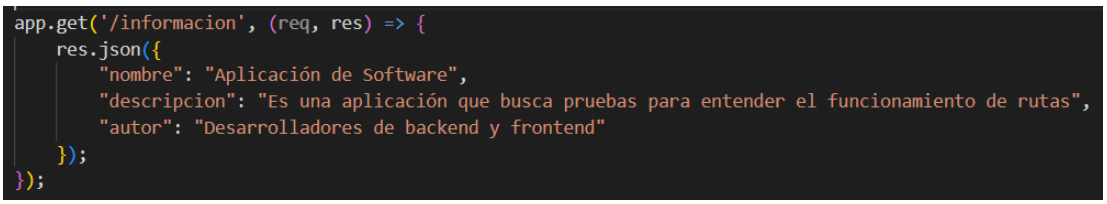
```
const express = require('express');
const app = express();
const PORT = 4000;
app.use(express.json());

app.get('/inicio', (req, res) => {
    res.json({ "mensaje de bienvenida": "Bienvenido a mi aplicación de API" });
});

```

Primero importé Express y creé mi aplicación utilizando express(). Después definí el puerto 4000, que es donde se va a ejecutar mi servidor. Luego creé la ruta GET /inicio, para que cuando alguien ingrese a esa dirección, mi aplicación responda con un mensaje de bienvenida en formato JSON.

4. **Crear una ruta GET /información que devuelva información general del servicio en formato JSON.**



```
app.get('/informacion', (req, res) => {
    res.json({
        "nombre": "Aplicación de Software",
        "descripcion": "Es una aplicación que busca pruebas para entender el funcionamiento de rutas",
        "autor": "Desarrolladores de backend y frontend"
    });
});
```

Después de crear la ruta de bienvenida, agregué otra ruta GET llamada /informacion. En esta parte coloqué el nombre de mi aplicación, una descripción y el autor. Utilicé res.json() para devolver toda esta información organizada en formato JSON.

5. **Crear una ruta POST /solicitudes que reciba nombre, apellido y correo electrónico en el cuerpo de la petición.**

```
app.post('/solicitudes', (req, res) => {
    const { nombre, apellido, correo } = req.body;
```

Después creé una ruta POST llamada /solicitudes, porque necesito recibir información que me envíe el usuario. Utilicé req.body para acceder a los datos que llegan en la petición y extraje el nombre, el apellido y el correo electrónico, que son los tres campos que me solicita la actividad.

6. **Validar que los tres campos sean enviados y que el correo tenga un formato razonable. Si los datos no cumplen, la API debe responder de forma clara.**

```
    if (!nombre || !apellido || !correo) {
        return res.status(400).json({
            error: "Datos incompletos",
            mensaje: "Los campos 'nombre', 'apellido' y 'correo' son obligatorios."
        });
    }
```

Después de recibir los datos, agregué una condición para comprobar que el usuario haya enviado los tres campos. Si falta el nombre, el apellido o el correo, mi API devuelve un error con el código 400 y un mensaje que explica que los campos son obligatorios.

7. **Cuando los datos sean válidos, responder que la solicitud fue recibida. No es obligatorio guardar datos en una base de datos.**

```
    return res.status(201).json({
        estado: "Éxito",
        mensaje: "¡La solicitud fue recibida correctamente!",
        datosRecibidos: {
            nombre,
            apellido,
            correo
        }
    });
});
```

Después de realizar las validaciones, agregué la respuesta para cuando todos los datos sean correctos. Utilicé el código 201 para indicar que la solicitud fue recibida correctamente y devolví un mensaje de éxito junto con los datos que ingresó el usuario. No guardé la información en una base de datos porque la actividad no lo exige.

8. **Incorporar al menos un middleware propio que registre en la consola el método HTTP y la ruta solicitada.**

```
app.use((req, res, next) => {
    console.log(`[LOG] Método: ${req.method} | Ruta solicitada: ${req.url}`);
    next();
});
```

Después agregué un middleware propio para observar las peticiones que llegan a mi servidor. Utilicé req.method para mostrar el método HTTP y req.url para mostrar la ruta solicitada. También utilicé console.log() para ver esa información en la terminal y next() para permitir que la petición continúe.

9. **Aplicar el middleware de análisis de JSON que ofrece Express, y justificar por qué debe registrarse antes de las rutas que leen req.body.**

```
app.use(express.json());
```

Agregué express.json() para que Express pueda interpretar los datos que llegan en formato JSON. Lo coloqué antes de las rutas porque necesito que los datos estén disponibles en req.body cuando el usuario envíe una solicitud POST. Así puedo leerlos y realizar las validaciones que hice anteriormente.

10. **Configurar respuestas para rutas inexistentes y para solicitudes con datos inválidos.**

```
app.use((req, res) => {
    res.status(404).json({
        error: "Ruta no encontrada",
        mensaje: `La ruta '${req.originalUrl}' no existe en este servidor.`
    });
});
```

Finalmente, agregué un middleware para manejar las rutas que no existen en mi aplicación. Si alguien intenta ingresar a una dirección que no he creado, mi servidor responde con el código 404 y muestra un mensaje indicando cuál fue la ruta solicitada. Las validaciones de los datos inválidos las realicé anteriormente en la ruta POST, donde devuelvo el código 400 si falta algún campo o el correo tiene un formato incorrecto.

3. Investigación obligatoria: middleware en Express

Antes de terminar la API, investiga qué es un middleware, para qué sirve y cómo participa en el procesamiento de una petición. Redacta las respuestas con tus propias palabras y agrega las fuentes consultadas.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

1. **¿Qué es un middleware en Express y en qué momento del ciclo de una petición se ejecuta?**

Un middleware es una función que se ejecuta cuando llega una petición al servidor. Sirve para realizar tareas como revisar datos, registrar información o comprobar permisos antes de continuar con la petición o enviar una respuesta.

2. **¿Qué significan los objetos req y res?**

- **req (request):** representa la petición que envía el cliente y contiene información como la ruta, el método HTTP y los datos enviados.
- **res (response):** permite al servidor enviar una respuesta al cliente, por ejemplo, un mensaje, datos en JSON o un código de estado.

3. **¿Cuál es la diferencia entre un middleware global, uno aplicado a una ruta específica y un middleware de manejo de errores?**

- **Middleware global**: se aplica a las peticiones que pasan por él, normalmente mediante app.use().
- **Middleware de ruta específica**: se ejecuta solamente en la ruta o las rutas donde se configura.
- **Middleware de manejo de errores:** sirve para capturar y gestionar errores durante el procesamiento de las peticiones. En Express se identifica porque recibe cuatro parámetros: err, req, res y next.

4. **¿Qué hace express?json() y qué problema se presenta si no se configura cuando el cliente envía JSON?**

express.json() es un middleware que permite leer los datos enviados en formato JSON y acceder a ellos mediante **req.body**. Si no lo configuro, los datos JSON pueden no estar disponibles para mi ruta **POST** y podría tener problemas al recibir y validar la información.

5. **¿Por qué importa el orden en que se registran los middleware y las rutas?**

El orden importa porque Express ejecuta los middleware según como los voy registrando. Por eso, debo colocar express.json() antes de las rutas que necesitan leer los datos y dejar el manejo de rutas inexistentes después de las rutas principales.

6. **¿Qué puede ocurrir si un middleware no envía una respuesta ni permite continuar el flujo?**

La petición puede quedarse esperando indefinidamente, porque el servidor no envía una respuesta ni continúa con el siguiente middleware. Para evitarlo, debo enviar una respuesta con res o utilizar next() para continuar el proceso.

7. ¿Cuál es la diferencia entre un middleware y la función que atiende una ruta?

El middleware realiza tareas durante el procesamiento de una petición, como registrar información o validar datos. La función que atiende una ruta se encarga de procesar una dirección específica y normalmente enviar la respuesta al cliente. Sin embargo, una función de ruta también puede cumplir funciones de middleware

8. ¿Qué riesgos o problemas podrían aparecer si una validación o una comprobación de acceso se omite?

Si no valido los datos, puedo recibir información incompleta, incorrecta o maliciosa. Si no compruebo los permisos, una persona podría acceder a información privada o realizar acciones que no le corresponden. Por eso, es importante validar los datos y verificar los permisos para proteger la aplicación.

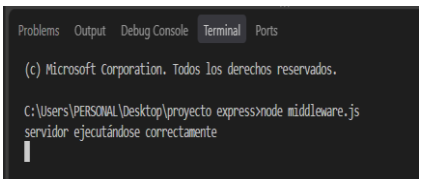
9. Incluye un ejemplo cotidiano, explicado sin código, que ayude a entender la función de un middleware.

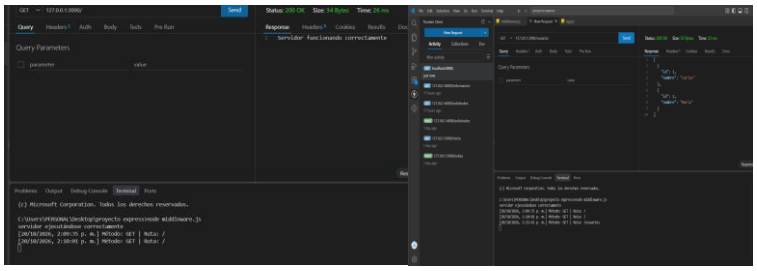
Un ejemplo sería el ingreso a una institución educativa. Primero, el vigilante revisa la identificación de la persona y comprueba si tiene autorización para entrar. Si cumple los requisitos, le permite continuar; si no, le impide el ingreso. El vigilante representa el middleware porque realiza una comprobación antes de permitir que el proceso continúe.

4. Construcción del middleware propio

Diseña e implementa un middleware propio que registre cada petición recibida. Como mínimo, debe permitir identificar el método HTTP (por ejemplo, GET o POST) y la ruta solicitada. Decide dónde ubicarlo y explica por qué elegiste esa ubicación.

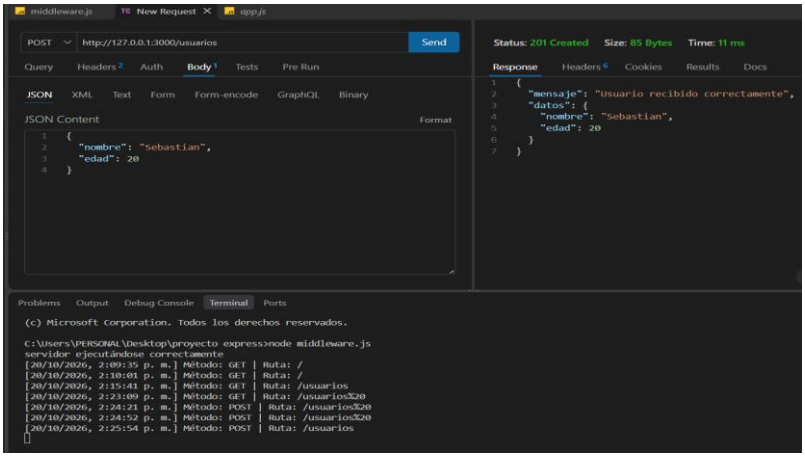
- **Comprueba que el registro se genere al consultar las rutas GET.**





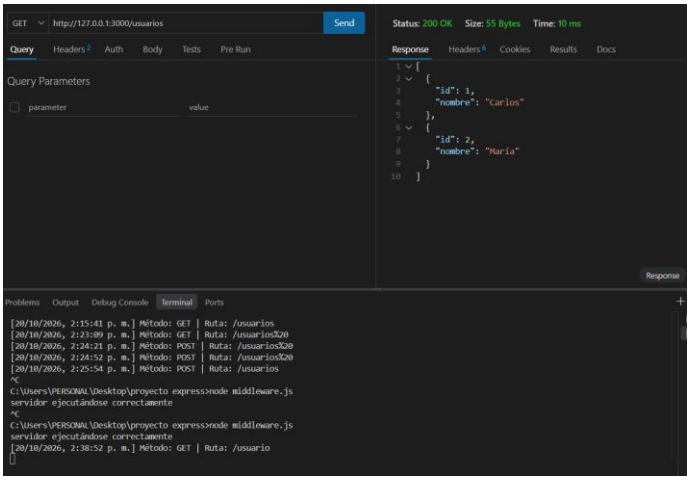
El middleware funciona correctamente, ya que registra las peticiones GET realizadas a las rutas / y /usuarios, mostrando la fecha, la hora, el método HTTP y la ruta solicitada en la consola. Además, se comprobó que el servidor responde correctamente y muestra la lista de usuarios.

- **Comprueba que el registro se genere al enviar una petición POST.**



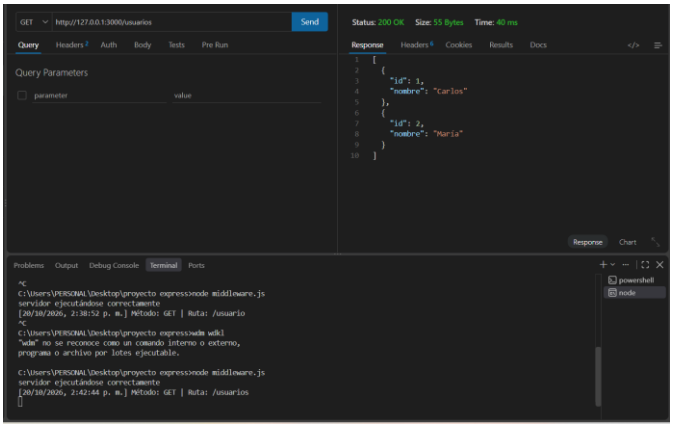
Al enviar la petición POST a la ruta /usuarios mediante Thunder Client, se comprobó que el middleware registra correctamente la solicitud en la consola de Visual Studio Code, mostrando la fecha, la hora, el método HTTP y la ruta solicitada. Además, el servidor responde con el estado 201 Created, confirmando que recibió correctamente los datos del usuario, como el nombre y la edad. Esto demuestra que el middleware funciona correctamente con las peticiones POST.

- **Explica qué cambia si el middleware se coloca después de las rutas en vez de antes.**



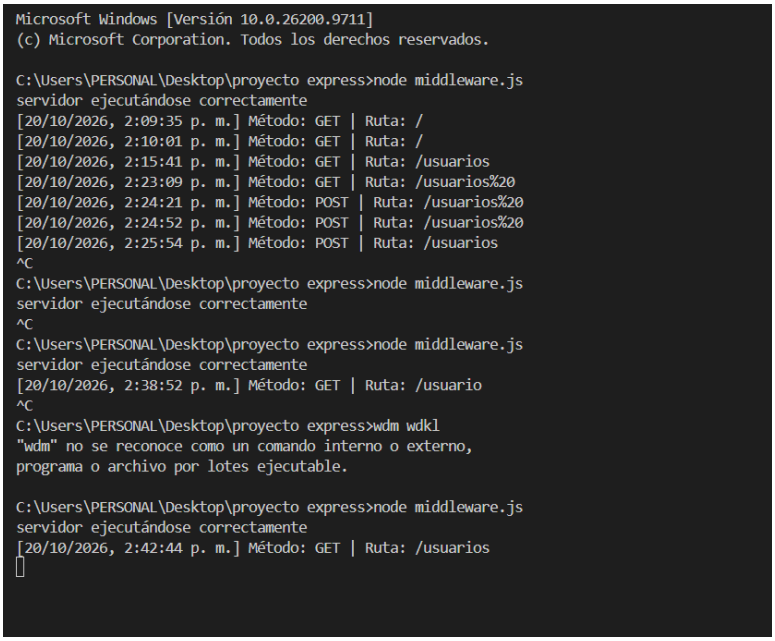
Al ubicar el middleware después de las rutas GET y POST, la petición a /usuarios devuelve correctamente la lista de usuarios, pero no genera un nuevo registro en la consola. Esto sucede porque la ruta envía la respuesta y finaliza la petición sin pasar el control al middleware siguiente. Por esta razón, es recomendable colocar el middleware antes de las rutas.

- **Describe cómo se transmite el control al siguiente middleware o a la ruta.**



La función next() permite pasar el control al siguiente middleware o a la ruta correspondiente. En este ejercicio, primero se registra la petición en la consola y luego next() permite que Express continúe el procesamiento y envíe la respuesta solicitada.

- **Adjunta capturas propias de la consola y de las pruebas realizadas.**



5. Códigos de estado HTTP y manejo de errores

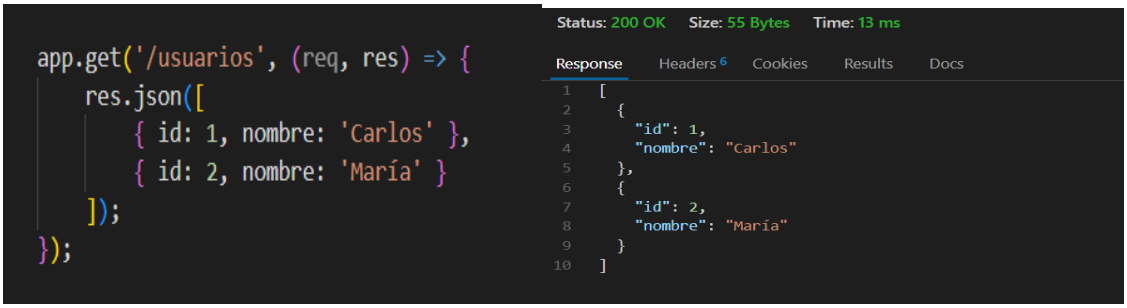
La API debe utilizar códigos de estado que correspondan con el resultado de cada operación. Investiga y aplica, como mínimo, los siguientes:

Código	Uso esperado
200 OK	La petición se procesó correctamente; por ejemplo, una consulta GET exitosa.
201 Created	La solicitud POST fue aceptada como una nueva solicitud o recurso creado, según el diseño que implementes.
400 Bad Request	El cliente envió datos faltantes, inválidos o con formato incorrecto.
404 Not Found	La ruta solicitada no existe.

500 Internal Server Error	Ocurrió un error inesperado del lado del servidor; investiga cuándo corresponde usarlo.

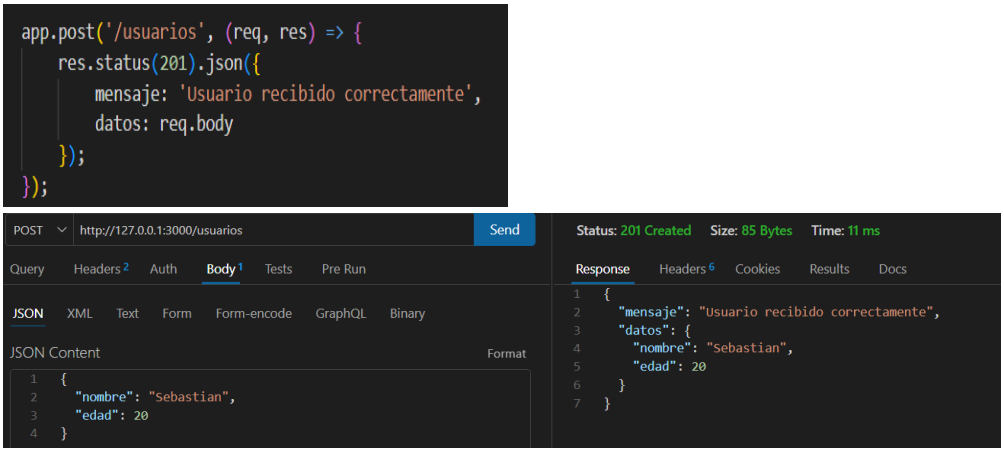
Para cada código, incluye en tu documento una explicación breve de por qué se utiliza y una evidencia de una prueba que lo produzca cuando aplique. No devuelvas un estado de éxito si la operación falló.

1 Código 200 OK — Consulta exitosa



El código 200 OK se utiliza cuando la petición se procesa correctamente. En esta prueba, el servidor devuelve la lista de usuarios solicitada y confirma que la consulta GET fue exitosa.

2 Código 201 Created — Petición POST exitosa



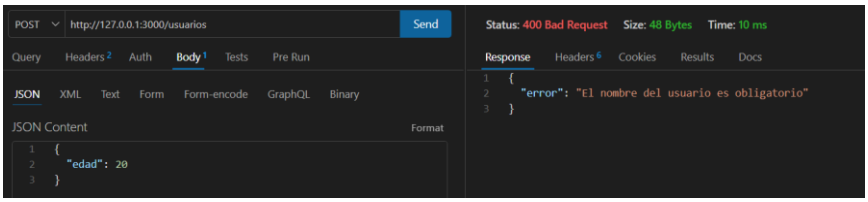
El código 201 Created indica que el servidor procesó correctamente la solicitud POST. En este ejercicio, confirma la recepción de los datos del usuario y devuelve la información recibida.

3 Código 400 Bad Request — Datos incorrectos

```
app.post('/usuarios', (req, res) => {
    const { nombre, edad } = req.body;

    if (!nombre || typeof nombre !== 'string' || !nombre.trim()) {
        return res.status(400).json({
            error: 'El nombre del usuario es obligatorio'
        });
    }

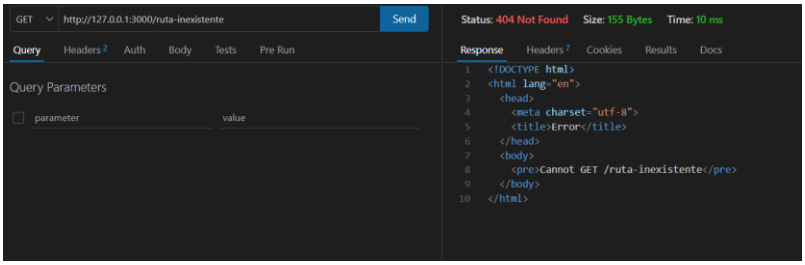
    res.status(201).json({
        mensaje: 'Usuario recibido correctamente',
        datos: { nombre, edad }
    });
});
```



El código 400 Bad Request se utiliza cuando el cliente envía datos incompletos o inválidos. En esta prueba se envía una petición POST sin el nombre del usuario, por lo que el servidor rechaza la solicitud y devuelve un mensaje de error en lugar de indicar que la operación fue exitosa.

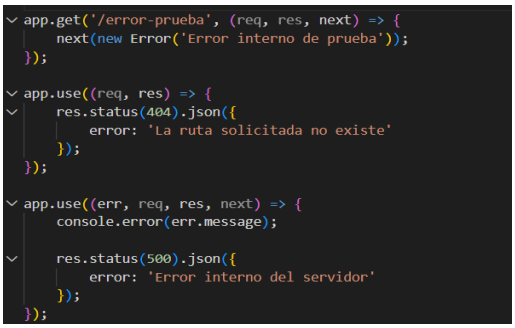
4 Código 404 Not Found — Ruta inexistente

```
app.use((req, res) => {
    res.status(404).json({
        error: 'La ruta solicitada no existe'
    });
});
```



El código 404 Not Found se utiliza cuando la ruta solicitada no existe en el servidor. En esta prueba se consulta una dirección que no está definida en la API y el servidor informa que no encuentra dicha ruta.

5 Código 500 Internal Server Error — Error del servidor

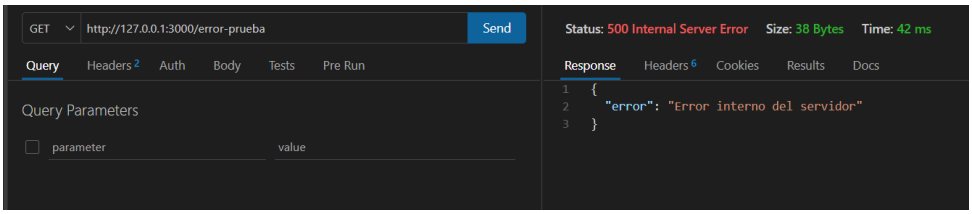


```
app.get('/error-prueba', (req, res, next) => {
    next(new Error('Error interno de prueba'));
});

app.use((req, res) => {
    res.status(404).json({
        error: 'La ruta solicitada no existe'
    });
});

app.use((err, req, res, next) => {
    console.error(err.message);

    res.status(500).json({
        error: 'Error interno del servidor'
    });
});
```



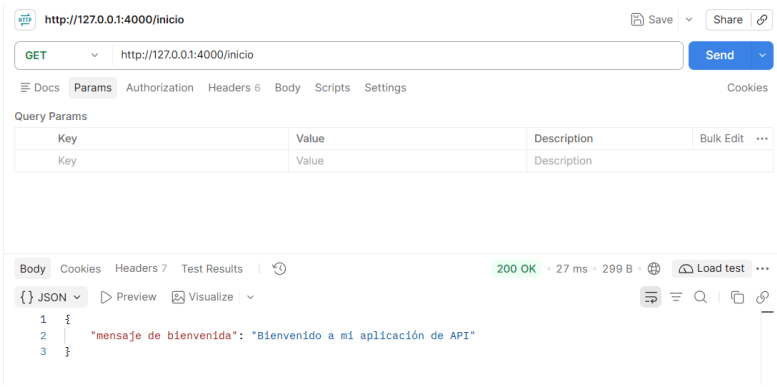
El código 400 Bad Request se utiliza cuando el cliente envía datos incompletos o inválidos. En esta prueba se envía una petición POST sin el nombre del usuario, por lo que el servidor rechaza la solicitud y devuelve un mensaje de error en lugar de indicar que la operación fue exitosa.

6. Pruebas de la API

Utiliza Postman, Insomnia o una herramienta equivalente para probar las rutas. Incluye capturas legibles en las que se vea el método, la URL, el cuerpo enviado cuando corresponda, el código de respuesta y el resultado.

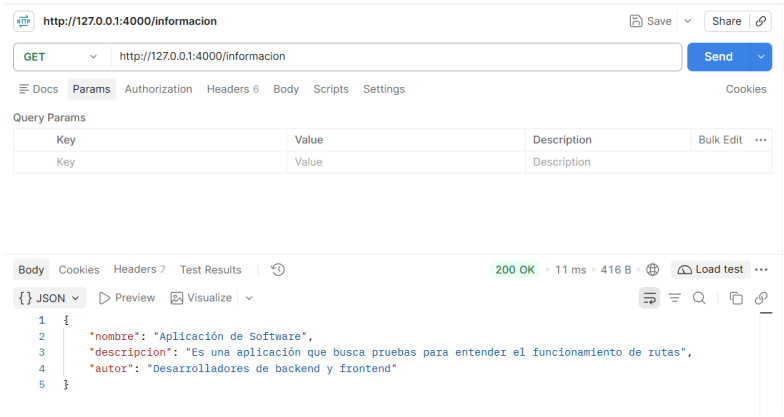
GET /inicio: verificar que devuelve una respuesta JSON.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



Se consultó la ruta /inicio mediante una petición GET y se obtuvo una respuesta JSON con un mensaje de bienvenida. El servidor respondió con el código 200 OK, confirmando que la ruta funciona correctamente.

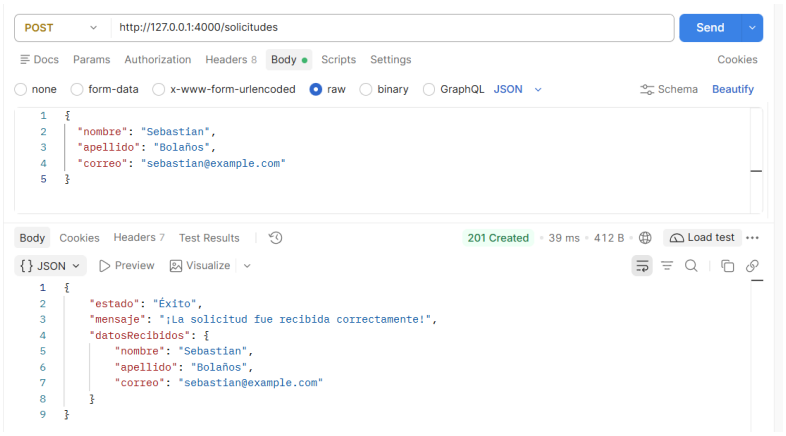
GET /informacion: verificar que devuelve la información general.



Se consultó la ruta /informacion y el servidor devolvió correctamente el nombre, la descripción y el autor de la aplicación en formato JSON, con el código 200 OK.

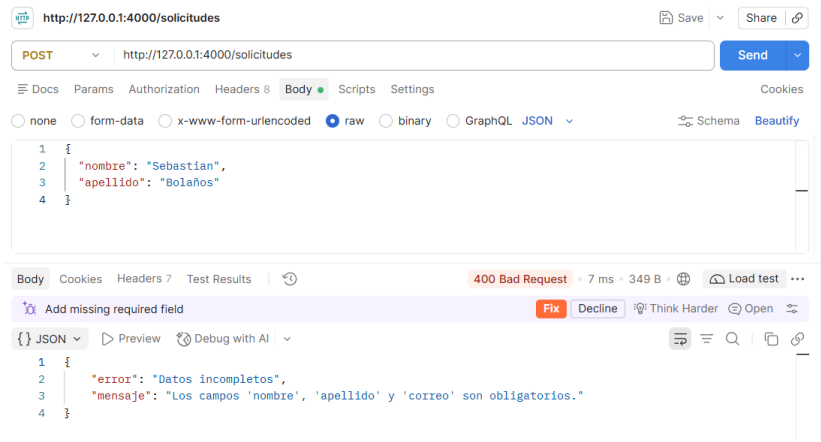
POST /solicitudes con nombre, apellido y correo válidos.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



Se envió una solicitud POST con nombre, apellido y correo. El servidor recibió los tres campos y respondió con el código 201 Created, confirmando que la solicitud fue procesada correctamente.

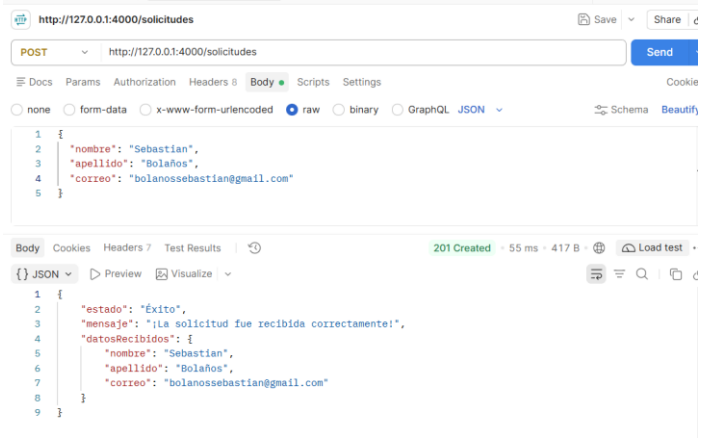
POST /solicitudes sin uno o más campos obligatorios.



Se envió una solicitud sin uno de los campos obligatorios. El servidor detectó que faltaba el correo y rechazó la petición con el código 400 Bad Request, indicando que los datos estaban incompletos.

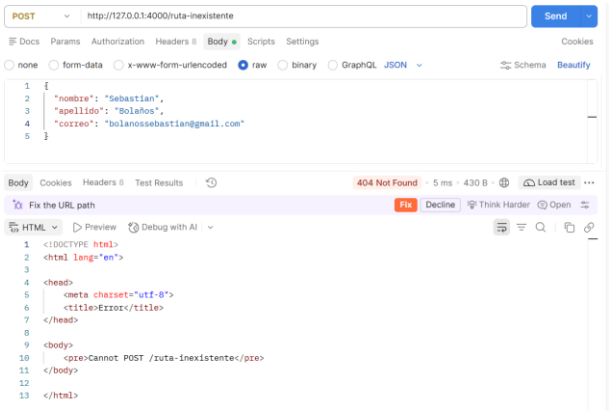
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

POST /solicitudes con correo de formato inválido.



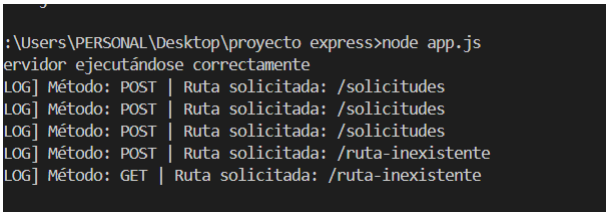
Se envió una solicitud con un correo electrónico de formato incorrecto. Después de implementar la validación, el servidor rechazó los datos y devolvió el código 400 Bad Request, evitando aceptar una solicitud inválida.

Consultar una ruta que no exista y verificar la respuesta 404.



Se consultó una ruta que no está definida en el servidor. El middleware de rutas inexistentes respondió con el código 404 Not Found e informó que la ruta solicitada no existe.

Verificar en la consola que el middleware propio registre las peticiones.



El middleware propio registra las peticiones recibidas y muestra el método HTTP y la ruta solicitada en la consola. La función next() permite continuar con el siguiente middleware o con la ruta correspondiente.

Probar qué ocurre si se envía un cuerpo JSON y explicar el papel de express.json().

```
app.use(express.json());
```

El middleware express.json() permite interpretar los datos enviados en formato JSON y almacenarlos en req.body. Esto facilita que el servidor lea y valide los campos recibidos en las peticiones POST.

7. Documento técnico obligatorio: paso a paso

Entrega un documento elaborado por ti que explique el proceso completo desde la instalación hasta las pruebas y la publicación del repositorio. No basta con enumerar comandos: debes explicar el porqué de cada decisión y qué sucedería si ese paso no se realiza.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

PASO O DECISIÓN	*¿POR QUÉ SE REALIZA?*	*¿QUÉ OCURRIRÍA SI SE OMITE?*
1. INSTALACIÓN DE NODE.JS Y NPM	Node.js permite ejecutar JavaScript en el servidor y npm facilita la instalación y administración de paquetes.	No podría ejecutar la aplicación con Node.js ni administrar fácilmente sus dependencias con npm.
2. CREACIÓN DE LA CARPETA Y APERTURA DEL PROYECTO	Permite organizar los archivos y trabajar de forma ordenada desde Visual Studio Code.	Los archivos podrían quedar desorganizados y sería más difícil encontrar y ejecutar el código correcto.
3. INICIALIZACIÓN DEL PROYECTO Y PACKAGE.JSON	Se utiliza npm init -y para crear el archivo que administra la información, las dependencias y los scripts del proyecto.	Sería más difícil administrar la configuración y compartir las dependencias del proyecto de forma organizada.
4. INSTALACIÓN DE EXPRESS	Se utiliza npm install express para facilitar la creación del servidor y el manejo de rutas y solicitudes HTTP.	Express no estaría disponible y habría que instalarlo antes de utilizar sus funciones.
5. INSTALACIÓN Y CONFIGURACIÓN DE NODEMON	Permite reiniciar automáticamente el servidor cuando se modifica el código durante el desarrollo.	Habría que detener y ejecutar nuevamente el servidor manualmente después de los cambios.
6. ARCHIVO PRINCIPAL Y DEFINICIÓN DEL PUERTO	El archivo app.js contiene la configuración de la aplicación. El puerto 4000 permite recibir las solicitudes de la API.	Si el servidor no se inicia o no escucha en el puerto correspondiente, Postman no podrá comunicarse correctamente con la aplicación.
7. MIDDLEWARE PARA ANALIZAR JSON	app.use(express.json()) permite interpretar los cuerpos JSON y acceder a sus datos mediante req.body.	Los datos enviados en JSON podrían no estar disponibles para las rutas como se espera.
8. MIDDLEWARE PROPIO	Registra el método HTTP y la ruta solicitada. Se coloca antes de las rutas	No se tendría el registro esperado de las peticiones. Si se coloca después de las

	para registrar las peticiones antes de que sean atendidas.	rutas que ya responden, podría no ejecutarse.
9. RUTAS GET Y POST	GET /inicio devuelve un mensaje de bienvenida, GET /informacion muestra información general y POST /solicitudes recibe los datos del usuario.	Las funcionalidades de las rutas que falten no estarán disponibles y las solicitudes correspondientes podrían devolver un error 404.
10. VALIDACIÓN Y CÓDIGOS DE ESTADO HTTP	Permiten comprobar los datos y comunicar el resultado: 200 para consultas exitosas, 201 para una operación POST exitosa, 400 para datos incorrectos, 404 para rutas inexistentes y 500 para errores internos.	El servidor podría aceptar datos incorrectos o informar un éxito cuando la operación realmente falló.
11. PRUEBAS CON POSTMAN	Permiten comprobar las rutas, los datos enviados, las respuestas y los códigos HTTP. También ayudan a detectar errores antes de entregar el proyecto.	Podrían pasar inadvertidos errores en las rutas, validaciones, respuestas o procesamiento del JSON.
12. GIT Y REPOSITORIO REMOTO	Git permite registrar los cambios y el repositorio remoto permite guardar y compartir el proyecto, por ejemplo, en GitHub.	Sería más difícil recuperar versiones anteriores, hacer seguimiento a los cambios y compartir una copia del proyecto.